

Two Pointers

Pattern



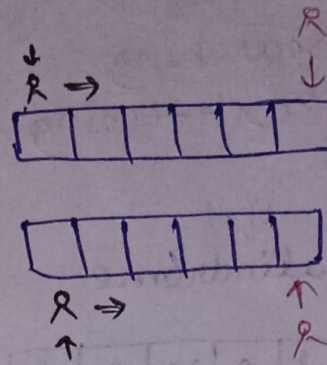
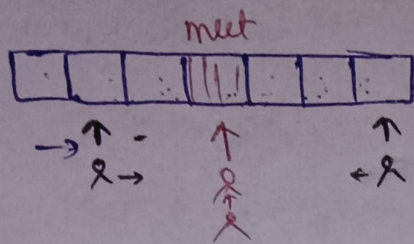
Handwritten Notes

by **Dipti Verma**

learned from **CTO Bhaiya**

Two Pointer

Day-1



Used in linear data structure $\Rightarrow$

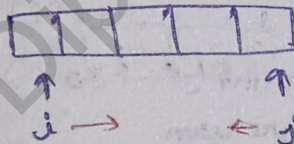
- $\rightarrow$ array
- $\rightarrow$ String
- $\rightarrow$ Linked list

A technique where two variables (pointers) traverse a data structure simultaneously from different positions

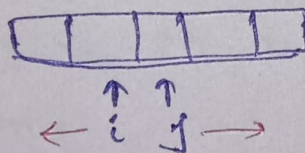
Character:

$\Rightarrow$ Two pointers either move towards each other, away from each other, or in the same direction.

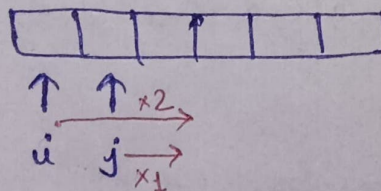
Towards



Away



Same / Fast-slow



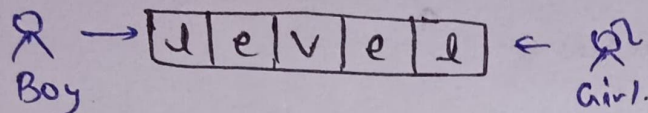
* Efficiently solving problems involving

- Comparisons
- Searching
- Partitioning

(PCS)

→ shortcut to learn

Example: Palindrome



Boy sees = level

Girl sees = level

> same = Palindrome ✓

Brute Force:

S = "level"

s_rev = s.reverse()

~~for~~ i = 0

while i < s.length:

if s[i] != s_rev[i]:

print("No")

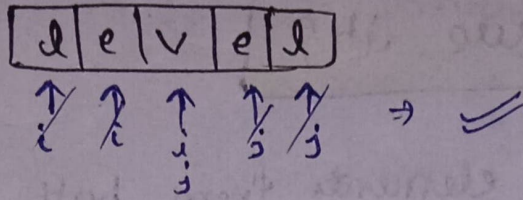
return

i = i + 1

print("Yes")

[TC = $O(n)$
SC = $O(n)$ because we use s_rev]

Optimization :



s = "level"

i = 0

j = s.length - 1

while i < j :

if s[i] != s[j] :

print("No")

return

i++

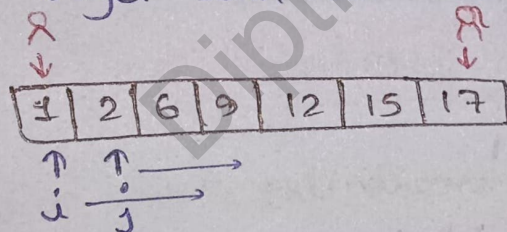
j--

print("Yes")

Why we use two pointers

→ No unnecessary iterations

eg Target Sum in sorted array

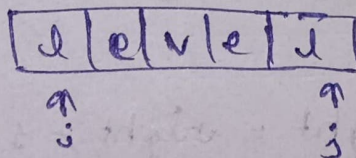


Brute Force = $O(n^2)$

@ two pointers = $O(n)$

→ Early stopping as soon as mismatch / condition fails

eg Palindrome



→ Works in-place → no need to create new arrays or copies

eg Palindrome

→ Improves both speed and space efficiency

Speed = $O(n^2) \Rightarrow O(n)$

Space = $O(n) \Rightarrow O(1)$

When to use it?

- Comparing elements from both ends (palindrome, pair sum)
- Finding pairs with specific properties (eg sum = target)
- Removing or merging elements (remove duplicates, merge sorted arrays)
- Detecting loops or middle elements ~~Fast-slow~~ (linked list cycle, middle node)

Template

① 1 Input and opposite side so done pointers

$a = [1, 4, 3, 2, 9]$

$left = 0$

$right = a.length - 1$

while $left < right$:

if condition :

$left = left + 1$

else :

$right = right - 1$

return ans

(merge array
in sorted array)

$$b = [9, 7, 6, 2, 0, 8]$$
$$\text{left} = 0$$

weight = 0

while left < a.length & right < b.length:

4) condition :

$$\text{left} = \text{left} + 1$$

cho

$$\text{eight} = \text{vight} + \text{I}$$

while left < a.length :

$$\text{left} = \text{left} + 1$$

while weight < b.length:

$$\text{weight} = \text{weight} + 1$$

eg, 1 4 6 9 10 —
↑ ↑ ↑ ↑ ↑
i i i i i

9 10 16 22 30

merge = [3, 4, 6, 9, 9, 10, 12]

10

here i exhaust n
we put j 's elements
directly

Day-2

Two Pointers Questions

Q. Leetcode valid palindrome (125)

eg S = "A man, a plan, a canal: Panama"

$\rightarrow$

(all match)

$\leftarrow$

Small
eg $\rightarrow$

S = $\begin{matrix} & a & p & & a \\ \nearrow & & \uparrow & \nwarrow & \nearrow \\ x & & i & & j \\ & & \uparrow & & \\ & & j & & \end{matrix}$

condition break

Code:

s = "1234567890"

i = 0

j = s.length() - 1

while (i < j) {

char l = s.charAt(i)

char r = s.charAt(j)

if (! Character.isLetterOrDigit(l)) {

i++

continue

}

if (! Character.isLetterOrDigit(r)) {

j--

continue

}

if (Character.toLowerCase(l) != Character.toLowerCase(r)) {

return false

}

i++

j--

}

return true;

Reverse String leetcode (344)

$s = ["h", "e", "l", "l", "o"]$

$\Rightarrow$

$i = 0$

$j = s.length - 1$

temp = h

~~s[j] =~~

while ($i < j$) {

char temp = s[i]

s[i] = s[j]

s[j] = temp

i = i + 1

j = j - 1

}

$TC = O(n)$
 $SC = O(1)$

Squares of a Sorted Array leetcode (977)

nums = [-4, -1, 0, 3, 10]

(non-decreasing order)

(square) [16, 1, 0, 9, 100]

↓ sort

[0, 1, 9, 16, 100]

$TC = O(n \log n)$

Now, improve T.C

Logic: nums = [-4, -1, 0, 3, 10]

[0, 1, 9, 16, 100]

↑ k


```
int result[] = new int[nums.length]
```

~~int~~

```
int i = 0
```

```
j = nums.length - 1
```

```
k = result.length - 1
```

```
while (i <= j) {
```

```
    l = Math.abs(nums[i])
```

```
    r = Math.abs(nums[j])
```

```
    if (l < r) {
```

```
        result[k] = (int) Math.pow(r, 2)
```

```
        j--
```

```
    } else {
```

```
        result[k] = (int) Math.pow(l, 2)
```

```
        i++
```

```
    }
```

```
    k--
```

```
}
```

```
return result
```

$TC = O(n)$

$SC = O(1)$

bez. result ko return kar rahi hai.

$i = j$

eg nums = [-4, -1, 0, 3, 10]

↑ ↑ ↑ ↑ ↑
l j 0 3 10

⇒ 0

result = [0, 1, 9, 16, 100]

⇒ 4 < 10

result = [, 100]

j--

⇒ 4 > 3

result = [, 16, 100]

⇒ 1 < 3

result = [9, 16, 100]

⇒ 1 > 0

result = [1, 9, 16, 100]

Q - Valid Palindrome II Leetcode (680)

Eg s = 'abbxa'

super power $\rightarrow$ delete at most 1 character & check palindrome or not.

$\nearrow$ $\uparrow$ $\uparrow$ $\searrow$
 i i j j
 b b x a
 not match
 (x delete)

a b b x a
 $\uparrow$ $\uparrow$
 i j
 match $\rightarrow$ Palindrome ✓

* [a b b x a

$\uparrow$ $\uparrow$
 i j
 (((Not match)))

two options $\Rightarrow$ shift+ (j or i) means delete (b or x)

$i++$ ✓

b x
 $\uparrow$ $\uparrow$
 i j

(not matched)

$j--$

b b
 $\uparrow$ $\uparrow$
 i j

(matched) ✓

x

Code:

```
public boolean palindromehelper(int i, int j, String s) {  
    while (i < j) {  
        if (s.charAt(i) != s.charAt(j)) {  
            return false;  
        }  
        i++;  
        j--;  
    }  
    return true;  
}
```

TC = $O(n)$

SC = $O(1)$

```
public boolean validPalindrome (String s) {  
    int i = 0;  
    j = s.length() - 1;
```

```
    while (i < j) {
```

```
        char left = s.charAt(i)
```

```
        right = s.charAt(j)
```

```
        if (left != right) {
```

```
            return palindromeHelper(i+1, j, s) ||
```

```
                palindromeHelper(i, j-1, s)
```

```
        } else {
```

```
            i++
```

```
            j--
```

```
        }
```

```
    }
```

```
    return true
```

```
}
```


Q - Valid Word Abbreviation (NeetCode)

Eg word = "apple", abbr = "a3e" → true

abbreviation of apple = a3e
(3)

Eg. w = ^{1 2 3 4 5 6 7 8 9 10 11 12} implementation

a = i 12 n
twelve

✓ → true

w = ^{1 2 3 4 5 6 7 8 9 10 11 12} implementation

a = i 12 n

a 5 7 e
↑ ↑
j j → 5
→ 5 × 10 + 7 = 57
curr × 10 + ('7' - '0')

num = 57

Eg w = a p p l e

a = a 4 0 e

i on 4th index

(length)

(not found index)

False

Code:

i = 0, j = 0

while (i < word.length() && j < abbr.length()) {

char a_c = abbr.charAt(j)

char w_c = word.charAt(i)

if (Character.isDigit(a_c)) {

if (a_c == '0') {

return false

int curr = 0

while (j < abbr.length() && Character.isDigit(abbr.charAt(j)))

curr = curr × 10 + (abbr.charAt(j) - '0');

j++


```

        i = i + curr
    } else {
        if (w-c != a-e) {
            return false
        }
        i++
        j--
    }
}

return i == word.length() && j == abbr.length();

```

TC = $O(m+n)$
 SC = $O(1)$

Day-3

Two Pointer Pattern

① Merge Sorted Array Leetcode (88)

nums1 = [1, 2, 3, 0, 0, 0, 0], m = 3

nums2 = [2, 5, 6, 7], n = 4

o/p = [1, 2, 2, 3, 5, 6, 7]

Brute force:

nums1 = [1, 2, 3, ~~0~~, ~~0~~, ~~0~~, ~~0~~]

nums2 = [~~2~~, ~~5~~, 6, 7]

and then sort the nums1 array.

→ But → TC = ~~$O((n+m) \log(n+m))$~~ $O((n+m) \log(n+m))$
 SC = $\log(n+m)$

Approaches

I $\Rightarrow$ $[1, 2, \underline{3}, 0, 0, 0]$
 $\downarrow \quad \downarrow \quad \downarrow$
 $i \quad i \quad i$
 Override hogya
 $[2, 5, 6, 7]$
 $\uparrow \quad \quad \quad \times$
 j

$1 < 2$ (do Nothing)
 $2 = 2$ ()
 $3 > 2$ (Put 2 in num1)

II $\Rightarrow$ $[1, 2, \underline{3}, 0, 0, 0]$ $3 > 2$
 $\downarrow \quad \downarrow \quad \downarrow$
 $i \quad i \quad i$
 $[2, 5, 6, 7]$
 $\downarrow \quad \downarrow$
 $j \quad j$
 Put 2 in num1 and shift 3 right side by 1
 = But TC = $O(n^2m)^2$

III $\Rightarrow$ $[1, 2, 3, 0, 0, 0, 0]$
 $\uparrow \quad \quad \quad \uparrow$
 $i \quad \quad \quad k$
 $[2, 5, 6, 7]$
 $\uparrow$
 j

* compare i & j and biggest element at k

① $[1, 2, 3, 0, 0, 0, 0]$
 $\downarrow \quad \quad \quad \uparrow$
 $i \quad \quad \quad k$
 $[2, 5, 6, 7]$
 $\downarrow$
 j

$3 < 7 \Rightarrow$

$[1, 2, 3, 0, 0, 0, 7]$
 $\downarrow \quad \quad \quad \downarrow$
 $i \quad \quad \quad k$
 $[2, 5, 6, 7]$
 $\downarrow$
 j

② $[1, 2, 3, 0, 0, 6, 7]$
 $\downarrow \quad \quad \quad \downarrow \quad \quad \quad \downarrow$
 $i \quad \quad \quad i \quad \quad \quad k$
 $[2, 5, 6, 7]$
 $\downarrow$
 j

③ $[1, 2, 3, 0, 5, 6, 7]$
 $\downarrow \quad \quad \quad \downarrow \quad \quad \quad \downarrow$
 $i \quad \quad \quad i \quad \quad \quad k$
 $[2, 5, 6, 7]$
 $\downarrow$
 j

$$\textcircled{1} \quad 3 > 2 \Rightarrow [1, \underline{2}, \cancel{3}, \overset{3}{\emptyset}, 5, 6, 7]$$

$\uparrow \quad \uparrow$
 $i \quad k$

$$[\underline{2}, 5, 6, 7]$$

$\uparrow$
 j

$$\textcircled{2} \quad 2 = 2 \Rightarrow [1, 2, \overset{2}{\cancel{\emptyset}}, 3, 5, 6, 7]$$

$\uparrow \quad \uparrow$
 $i \quad k$

fill num's
2 in num's

$$[2, 5, 6, 7]$$

$\uparrow$
 j

$$\textcircled{3} \quad 1 < 2 \Rightarrow [1, \overset{2}{2}, 2, 3, 5, 6, 7]$$

$\uparrow \quad \uparrow$
 $k \quad$

$$[2, 5, 6, 7]$$

$\uparrow$
 j

J par ho gya $\rightarrow$ end.

$$\text{o/p} = [1, 2, 2, 3, 5, 6, 7]$$

Code 8

```

i = m - 1
j = n - 1
k = m + n - 1

```

```

while ( i >= 0 && j >= 0 ) {
    if ( nums1[i] < nums2[j] ) {
        nums3[k] = nums1[i]
        i--
    } else {
        nums3[k] = nums2[j]
        j--
    }
    k--
}

```

TC = $O(m+n)$

SC = $O(1)$

11 pending elements in j

```

while ( j >= 0 ) {
    nums3[k] = nums2[j]
    j--
    k--
}

```

Q Count Pairs whose sum is less than target (2824)

Eg $nums = [-1, 1, 2, 3, 1]$

target = 2

• $-1 + 1 = 0 < 2 \rightarrow \text{count} = 1$

• $-1 + 2 = 1 < 2 \rightarrow \text{count} = 2$

• $-1 + 3 = 2 < 2 \times$

• $-1 + 1 = 0 < 2 \rightarrow \text{count} = 3$

• $1 + 2 = 3 < 2 \times$

• $1 + 3 = 4 < 2 \times$

• $1 + 1 = 2 < 2 \times$

Brute Force

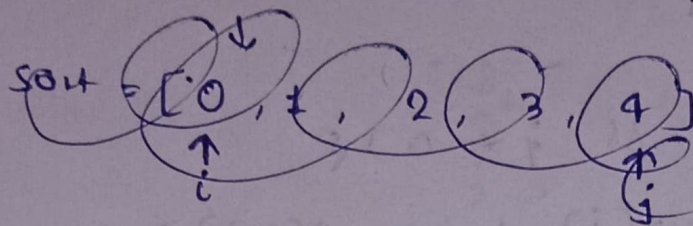
count = 3

TC = $O(n^2)$

SC = $O(1)$

Optimized

eg nums = [3, 1, 0, 4, 2] ; target = 5



pairs:

0, 1	1, 2	2, 3	3, 4
0, 2	1, 3	2, 4	
0, 3	1, 4		
0, 4			

$$0 + 4 \Rightarrow 4 < 5 \quad \checkmark$$

means, uske upar vale bhi pairs satisfy honge

$$1 + 4 \Rightarrow 5 < 5 \quad \times$$

then check uske upar

$$1 + 3 \Rightarrow 4 < 5 \quad \checkmark$$

mean, uske upar vale pairs = satisfy

sort the nums = [0, 1, 2, 3, 4]

index $\Rightarrow$ 0 1 2 3 4

↑ ↑ ↑ ↑ ↑

i i i j j

$$\Rightarrow 0 + 4 = 4 < 5 \quad \checkmark \rightarrow \text{pair} = 4 \Rightarrow (j - i)$$

i++

$$\Rightarrow 1 + 4 = 5 < 5 \quad \times$$

j--

$$\Rightarrow 1 + 3 = 4 < 5 \quad \checkmark = \text{pair} = j - i = 3 - 1 = 2$$

i++

$$\Rightarrow 2 + 3 = 5 < 5 \quad \times$$

j--

① & ④ at same $\rightarrow$ condition $\checkmark \Rightarrow$ stop

Total pair = $4 + 2 = \underline{6}$

Code :

```
public int countPairs( List<Integer> nums, int target ) {
    Collections.sort(nums);
    int i = 0;
    int j = nums.size() - 1;
    int count = 0;
    while ( i < j ) {
        int sum = nums.get(i) + nums.get(j);
        if ( sum < target ) {
            count = count + (j - i);
            i++;
        } else {
            j--;
        }
    }
    return count;
}
```

TC = $O(n \log n)$

SC = $O(\log n)$

① - Two Sum Leetcode (1)

eg. nums = [2, 7, 11, 15]

target = 9

TC = $O(n^2)$
SC = $O(1)$

Brute Force :

(2, 7) 7, 11 11, 15
2, 11 7, 15
2, 15

(0, 1) output

Optimized:

[2, 7, 11, 15]
↑ ↑ ↑ ↑
j i i i

target = 26 TC = $O(n)$
SC = $O(n)$

Set

value	index
2 : 0	
7 : 1	
11 : 2	

at index 3 :
ask (26 - 15 = 11)
is 11 in Hash set
Yes
return index

at index 0:

ask : (26 - 2 = 24)

is 24 in Hash set ?

→ No, then put 2 in
with index.

Code :

```
Map< Integer, Integer> map = new HashMap<>();  
for ( int i = 0 ; i < nums.length ; i++ ) {  
    int lookingFor = target - nums[i]  
    if ( map.containsKey ( lookingFor ) ) {  
        return new int[] { i , map.get(lookingFor) } ;  
    }  
    map.put(nums[i] , i) ;  
}  
  
return new int[] { -1 , -1 } ;
```

Q. Two Sum II - Input Array is Sorted (167)
[Here, don't use extra spaces]

numbers = [1, 3, 4, 5, 7, 10, 11] target = 9

(use the concept of Count pair)

[1, 3, 4, 5, 7, 10, 11]
 ↑ ↑ ↑ ↑ ↑ ↑
 i i i j j j

• $1 + 11 = 12 > 9 \rightarrow j--$

$4 + 10 = 14 > 9 \rightarrow j--$

$1 + 7 = 8 < 9 \rightarrow \underline{i++}$

• $3 + 7 = 10 > 9 \rightarrow j--$

$3 + 5 = 8 < 9 \rightarrow i++$

• $4 + 5 = 9 = 9 \rightarrow \underline{\text{return } i \text{ and } j}$

Code :

$i = 0$

$j = \text{numbers.length} - 1$

while ($i < j$) {

$\text{sum} = \text{numbers}[i] + \text{numbers}[j]$

 if ($\text{sum} > \text{target}$) {

$j--$

 } else if ($\text{sum} < \text{target}$) {

$i--$

 } else {

 return new int[] { $i+1$, $j+1$ } ;

 }

}

return new int[] { -1, -1 }

① - 3Sum (15)

nums = [-1, 0, 1, 2, -1, -4] target = 0

* return triplet value * do not add duplicate triplet.

Sort: [-1, -1, -4, 0, 1, 2]

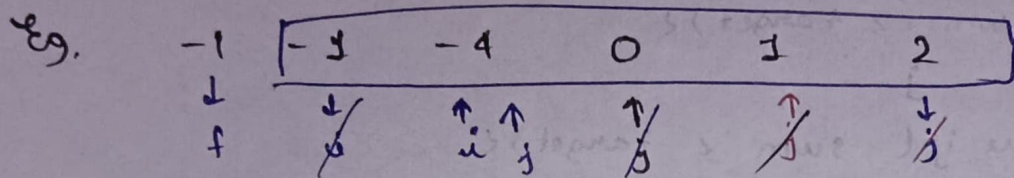
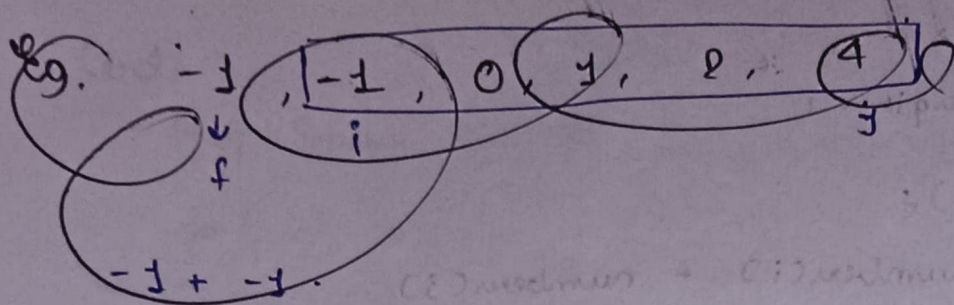
↓ ↓ ↓
f i j

$$-1 + (-1 + 2) = \text{target}$$

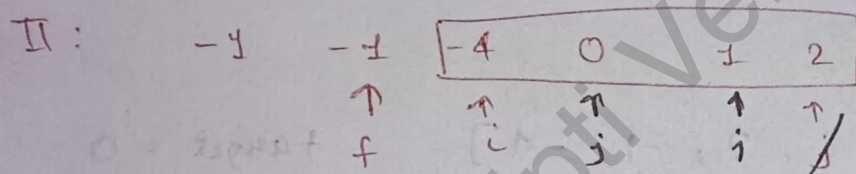
in next iteration:

-1, -1, -4, 0, 1, 2

↓ ↓ ↓
f i j



I: $-1 + -1 + 2 = 2 \neq 0 \rightarrow j--$ return $[-1, -1, 2]$
 $-1 + -1 + 1 = 1 \neq 0 \rightarrow j--$
 $-1 + -1 + 0 = 0 = 0 \checkmark$ $[-1, -1, 0]$
 $i++$, $j--$
 break.

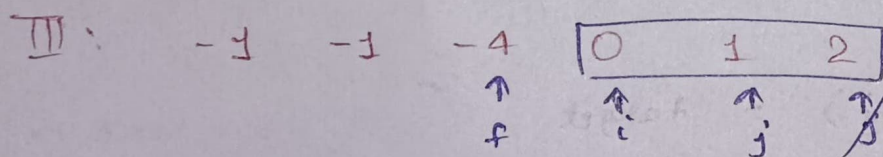


$-1 + -4 + 2 \neq 0 \rightarrow j--$

$-1 + -4 + 1 \neq 0 \rightarrow j--$

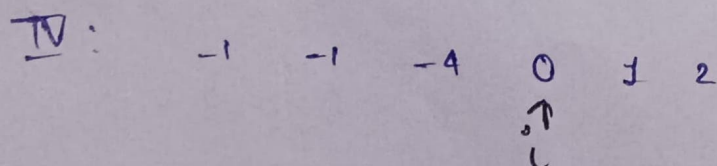
$-1 + -4 + 0 \neq 0 \rightarrow j--$

Not too
do this.
bcz,
hamane (-1)



$-4 + 0 + 2 \neq 0 \rightarrow j--$

$-4 + 0 + 1 \neq 0 \rightarrow j--$ break.



Now, here if we add something in $0 \neq 0$
 So, stop.

One Move
Case

$[-1, 0, 0, 1, 1]$

↓ ↓ ↑ ↑ ↓
f i j j j

target = 0

$-1 + 0 + 1 = 0 = \text{target} \rightarrow [-1, 0, 1]$

i++
j--

$-1 + 0 + 1 = 0 = \text{target} \rightarrow [-1, 0, 1] \times$

No duplicate
triplets



Toh frame, i ko ++ karna hai jab tak unique value
nhi mile.
or j ko --

TC = $O(n \log n + n^2)$

SC = $O(\log n) \rightarrow$ bcz, we use
sort.

Code ? class solution

```
public void twoSumHelper(int f, int nums[], List<List<Integer>> res){
```

```
    int i = f + 1
```

```
    j = nums.length - 1
```

```
    while (i < j){
```

```
        sum = nums[f] + nums[i] + nums[j]
```

```
        if (sum > 0){
```

```
            j = j - 1
```

```
        } else if (sum < 0){
```

```
            i = i + 1
```

```
        } else {
```

```
            // found a triplet
```

```
            res.add(Arrays.asList(nums[f], nums[i],  
                                   nums[j]));
```

```
            i++
```

```
            j--
```

unique value $\rightarrow$ while (i < j && nums[i] == nums[i+1]) {

i++

while (i < j && nums[j] == nums[j+1]) {

j--


```

public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();

    for (int f=0; f < nums.length; f++) {

```

```

        if (nums[f] > 0) {

```

```

            break;

```

```

        if (f == 0 || nums[f] != nums[f-1]) {
            twoSumHelper(f, nums, res)

```

```

        }
    }
    return res;
}

```

((Day - 4)) [Two Pointer Pattern]

(3 Popular Interview Q)

Q. Sort two colors

⇒ 2 elements: 0 & 1

nums = [0, 1, 1, 1, 0, 0, 1, 1]

o/p = [0, 0, 0, 1, 1, 1, 1, 1]

Brute force

• count '0' = 3

• count '1' = 5

• 000 11111 (by loop)

TC = $O(n^2)$

① n = For count
② n = for

→ 4 lines Two Iteration

1st Count

2nd Over write

Optimized

☆ ☆ ☆ Concept
i ke piche nara $\Rightarrow 0$
j ke piche nara $\Rightarrow 1$

nums = [0, 1, 1, 1, 0, 0, 1, 1]
 ↑ ↑ ↑
 i i j

• $j \neq 0 \checkmark \rightarrow i++$

gese • $j = 1$ and $j = 1$

swap i & j

→ it confirm that last index should have

$\rightarrow j--$ & $i++ \times$ bec $j \neq 0$.

nums = [0, 1, 1, 1, 0, 0, 1, 1]
 ↑ ↑ ↑ ↑ ↑
 i i j j j

• $j = 1$ & $j = 1$

swap & $j--$

• $j = 1$ & $j = 0$

swap & $j--$ & $i++$ bec $i = 0$

nums = [0, 0, 1, 1, 0, 1, 1, 1]
 ↑ ↑
 i j

$i = 1$ & $j = 0$

do same

[0, 0, 0, 1, 1, 1, 1, 1]
 ↑ ↑ ↑
 i j j

$\Rightarrow j = 1$ then $j--$

here

i ke piche nara = 0
(phale)

j ke piche nara = 1

Code:-

```
j = 0
j = nums.length - 1
while ( i <= j ) {
    if ( nums[i] == 0 ) {
        i++
    } else {
        temp = nums[i]
        nums[i] = nums[j]
        nums[j] = temp
        j--
    }
}
```

TC = $O(n)$

SC = $O(1)$

Here we use only
1 Iteration.

Q. Sort Colors Leetcode (75)

Three color = 0, 1, 2

This problem is also called
"Dutch National Flag Problem"

Eg nums = [2, 0, 2, 1, 1, 0]

o/p = [0, 0, 1, 1, 2, 2]

nums = [1 2 0 0 1 2 2 0 1]

o/p = [0 0 0 | 1 1 1 | 2 2 2]
 i j k

① says i ke piche sara = 0

② says j ke aage sara = 2

③ says k ke piche sara = 1 i tak

concept

$\Rightarrow$

1	2	0	0	1	2	2	0	1
$\uparrow$	$\uparrow$						$\uparrow$	
k	k						j	

$k=1 \checkmark \Rightarrow k++$

$k=2 \times \Rightarrow$ ② toh j ko chahiye

toh swap (k, j) and j--

Here, $k++ \times$ bcz, j mein koi bhi value ho sakti hai

num =

1	0	1	0	0	1	2	2	0	2
$\uparrow$		$\uparrow$	$\uparrow$					$\uparrow$	
j		k	k					j	

$k=1 \checkmark \Rightarrow k++$

$k=0 \times \Rightarrow$ ① toh i ko chahiye
toh swap (i, k) and i++

[Yaha peh 2 cases ho skte hain]

$k=i$

i & k are at same position

and $k=0$

then

$i++$ & $k++$

k, i ke aage baitha ho

$k > i$

and $k=0$

then

Obviously $k=1$

so, we $k++$ surely...

num =

0	1	0	1	0	2	2	0	2
$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	
k	i	k	k	k	k	j	j	

• $k=0 \times \Rightarrow$ swap with i

• $k=1 \rightarrow k++$

• $k=2$ and swap with j and j--

• $k=0 \Rightarrow$ swap with i

num = 0 0 0 1 1 1 2 2 2

$\uparrow$ $\uparrow$ $\uparrow \downarrow$
i *j* *k*/*j*

here, $k > 2$ swap with *j*
and *j* -- (stop)

o/p get :

0	0	0
---	---	---

1	1	1
---	---	---

2	2	2
---	---	---

$\uparrow$ $\uparrow$ $\uparrow$
i *j* *k*

Code:

i = 0

j = num.length - 1

k = 0

TC = $O(n)$

SC = $O(1)$

while (*k* <= *j*) {

if (num[*k*] == 1) {
 k ++

} else if (num[*k*] == 2) {

~~temp = num[*k*]~~
 swap (num[*k*] , num[*j*])

j --

} else {

 swap (num[*k*] , num[*i*])

i ++

k ++

}

nums = 0 0 0 1 1 1 2 2 2

$\uparrow$ $\uparrow$ $\uparrow \uparrow$
i *j* *k/j*

here, $k=2$ swap with *j*
and *j*-- (stop)

o/p get :

0	0	0
---	---	---

1	1	1
---	---	---

2	2	2
---	---	---

$\uparrow$ $\uparrow$ $\uparrow$
i *j* *k*

Code:

$i = 0$

$j = \text{nums.length} - 1$

$k = 0$

TC = $O(n)$

SC = $O(1)$

while ($k \leq j$) {

if ($\text{nums}[k] == 1$) {

$k++$

} else if ($\text{nums}[k] == 2$) {

~~temp = nums~~

swap ($\text{nums}[k]$, $\text{nums}[j]$)

$j--$

} else {

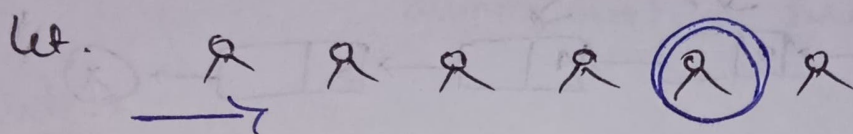
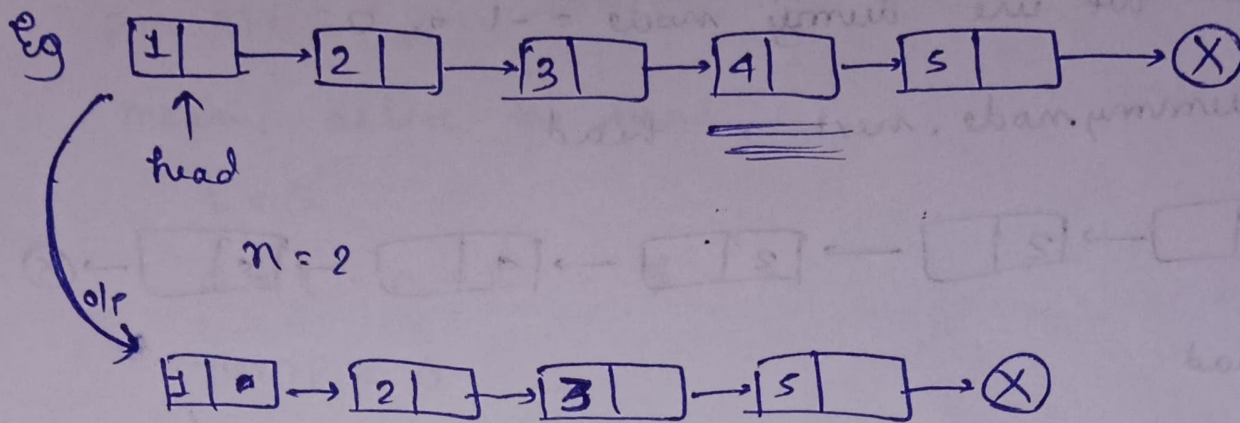
swap ($\text{nums}[k]$, $\text{nums}[i]$)

$i++$

$k++$

}

Q. Remove Nth Node From End of List (19)



Humne piche se dueno (2) child ko marna hai
aur hum bas aage me jaa rkte hai

Toh kya karenge ???

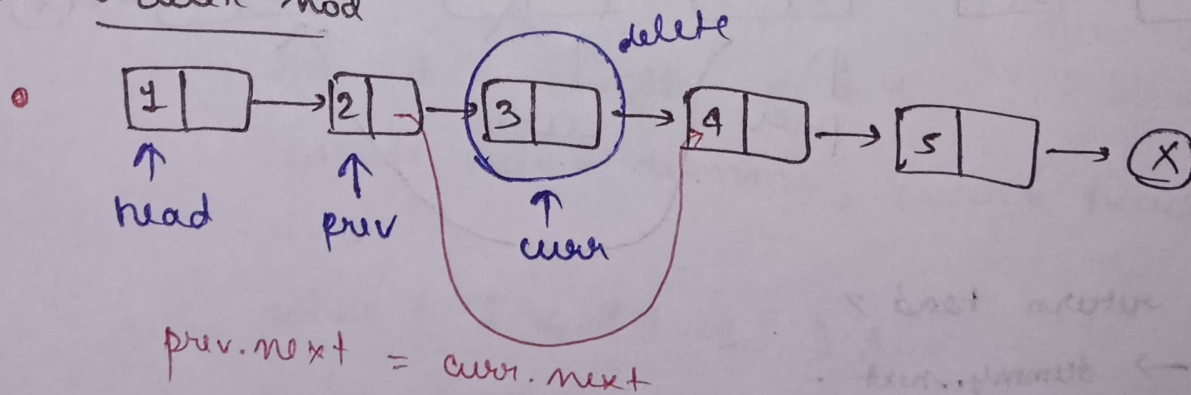
Total child = 6

Hit child = 2

$$\rightarrow 6 - 2 + 1 = 5$$

toh aage se 5th child ko marna hai !!!

How to delete node

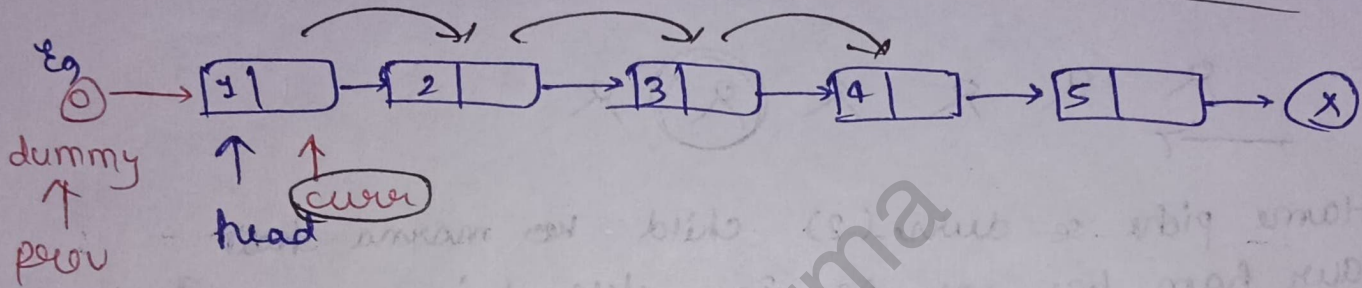
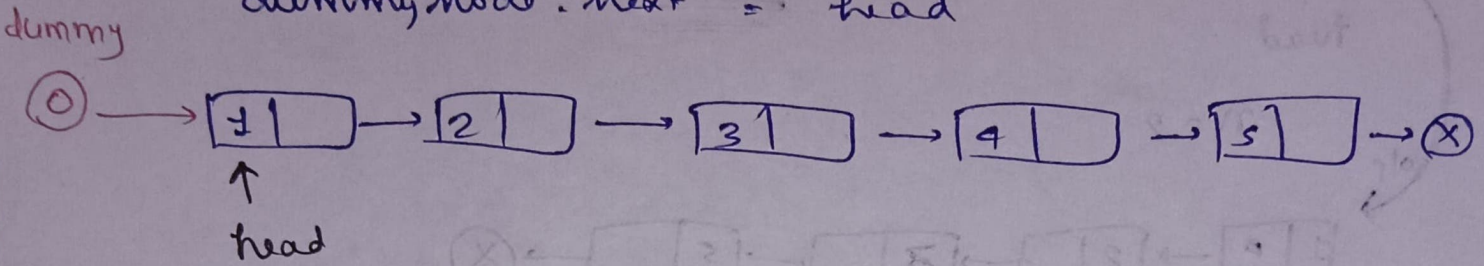


$$\Rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow (X)$$

4) we want delete a head?

→ we use dummy^m node = -1 or 0

dummy node . next = head

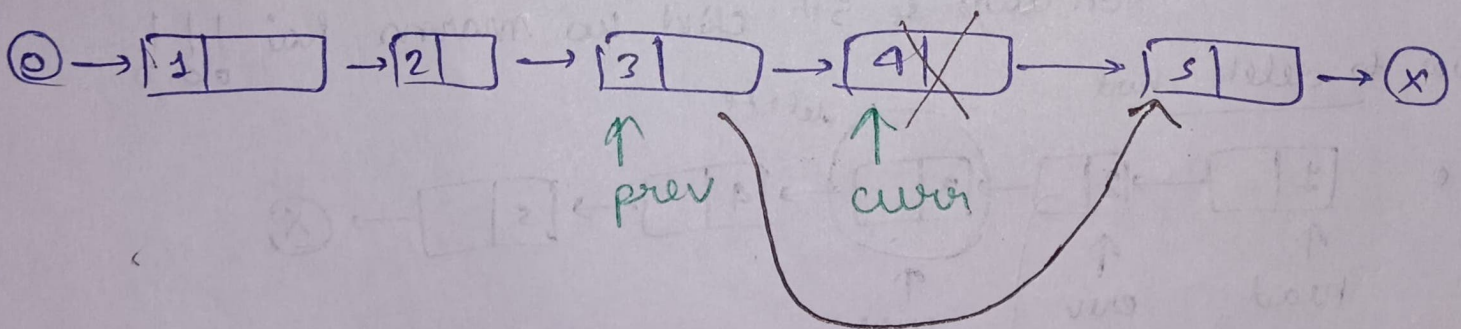


$$n = 2$$

$$t = 5$$

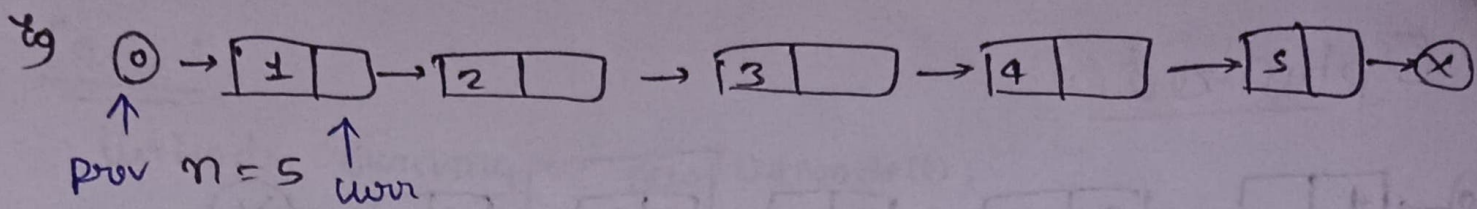
$$d = 4 \quad (5 - 2 + 1)$$

curr ko 3rd times jump karwana hai to delete 4
also, do same for prev



How to return head?

→ dummy . next .



means, delete the head

$$t = 5$$

$$d = t - n + 1 = 1$$

$$\text{jump} = 0$$

→ direct : $\text{prev.next} = \text{curr.next}$
 $\text{dummy.next} = \text{head}$

Code:-

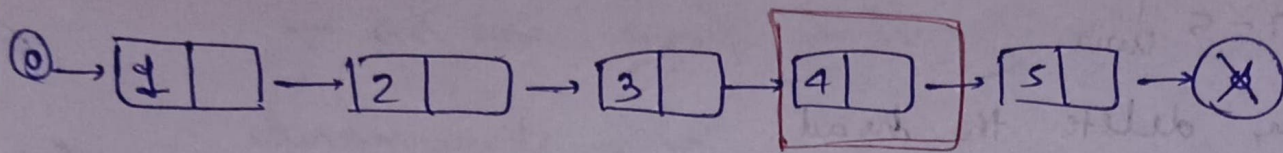
```

public ListNode removeNthFromEnd ( ListNode head, int n) {
    ListNode dummy = new ListNode(0)
    dummy.next = head
    int len = 0
    ListNode length = head
    while ( length != null) {
        len = len + 1
        length = length.next
    }
    int d = len - n + 1
    ListNode prev = dummy, curr = head
    i = 0
    while ( i < (d - 1) ) {
        curr = curr.next
        prev = prev.next
        i++
    }
    prev.next = prev.next.next
    return dummy.next
}
  
```

$T.C = O(n)$
 $S.C = O(1)$

here we use 2 iteration

Optimized



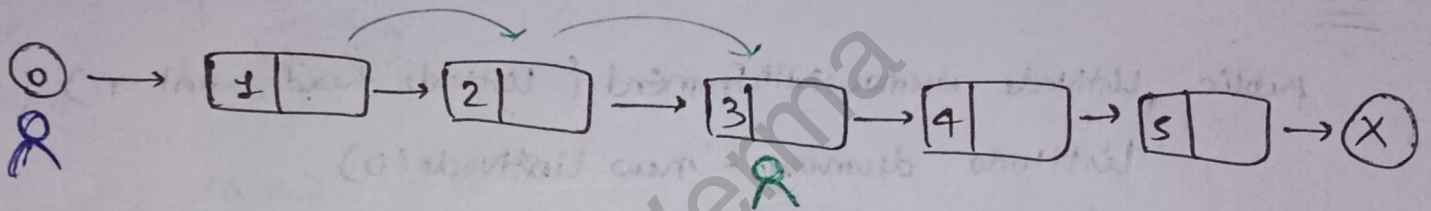
$n=2$

$\Rightarrow$ only given $n=2$

pithe se ②

age se ④ > jump

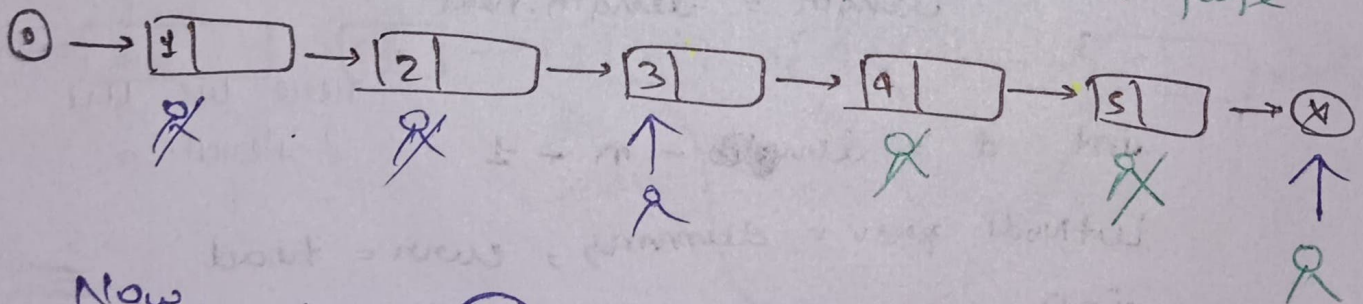
think



- $n=2 \rightarrow R \Rightarrow \text{jump} = 2$ from head
- R at dummy

Now jump both R together

jab tak $R \rightarrow \text{null}$ pe nah aa jaye



Now, see $R \text{ .next} = 4 \rightarrow$ which we want to delete

Code :

```
ListNode dummy = new ListNode(0);  
dummy.next = head
```

```
ListNode l = head  
int jumps = 0
```

```
while (jumps < n && l != null) {
```

```
    l = l.next  
    jumps++  
}
```

do 2 jumps

```
ListNode prev = dummy, curr = l;
```

```
while (curr != null) {  
    curr = curr.next  
    prev = prev.next  
}
```

at null

at deleted-node → prev

```
prev.next = prev.next.next // delete the node at n=2
```

```
return dummy.next
```

This done in 1 Iteration

Day-5

Two Pointer Pattern

(Interview Questions)

Q. Strobogrammatic Number

Number ko 180° rotate karne ke baad same dekhe jese

0, 1, 8, 6, 9

rotate (S = "198861") same
 ↓ ↓ ↓ ↓ ↓ ↓
 198861

strobogrammatic no. ✓

Let 2 → 7 → This is not strobo no.

Eg. "6 9" $\xrightarrow[180^\circ]{\text{rotate}}$ "9 6" → SN ✓

Code:

```
ListNode dummy = new ListNode(0);  
dummy.next = head
```

```
ListNode l = head  
int jumps = 0
```

```
while (jumps < n && l != null) {
```

```
    l = l.next  
    jumps++
```

do 2 jumps

```
ListNode prev = dummy, curr = l;
```

```
while (curr != null) {
```

```
    curr = curr.next
```

```
    prev = prev.next
```

at null

at deleted node → prev

```
prev.next = prev.next.next // delete the node at n
```

```
return dummy.next
```

This done in 1 Iteration

Day-5

Two Pointer Pattern (Interview Questions)

Q. Strobogrammatic Numbers

Numbers ko 180° rotate karne ke baad same dekhne jese

0, 1, 8, 6, 9

rotate $\left(\begin{array}{c} S = "198861" \\ \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \\ 198861 \end{array} \right)$ same

strobogrammatic no. ✓

Let 2 → 7 → This is not strobo no.

eg. "69" $\xrightarrow[180^\circ]{\text{rotate}}$ "69" → SN ✓

eg. $S = "198861"$

rotate
 $S = "198861"$
 $S = "198861"$

in general

198861

$1 \leftrightarrow 1$

$6 \leftrightarrow 9$

$9 \leftrightarrow 6$

$8 \leftrightarrow 8$

$S = 198861$

$\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$
 $i \quad j \quad i \quad j \quad i \quad j$

$1 \leftrightarrow 1 \checkmark \quad i++, j--$

$9 \leftrightarrow 6 \checkmark \quad i++, j++$

$8 \leftrightarrow 8 \checkmark$

S.N $\checkmark$

WHO SN NOT?

2, 3, 4, 5, 7

eg.

$S = 168891$

Store in Map

0	→	0
1	→	1
8	→	8
6	→	9
9	→	6

• at $i = 1$

→ 1 found in Map

→ 1 ka value map mein 1

→ and $j = \text{value} = 1$

→ So, $i++$ & $j--$

Case: $\begin{matrix} \downarrow & & \downarrow \\ 1 & 2 & 1 \\ \nearrow & \uparrow & \searrow \end{matrix}$

2 → not contain in Map

So, SN X

Case: $\begin{matrix} \downarrow & & \downarrow \\ 1 & 6 & 6 & 8 \\ \nearrow & & \searrow \end{matrix}$

i & j not match with map

X

Code: String s = "198861"

```
Map<Character, Character> map = new HashMap<>();  
map.put('0', '0');  
map.put('1', '1');  
map.put('8', '8');  
map.put('6', '9');  
map.put('9', '6');
```

TC = $O(n)$

SC = $O(1)$

Method

```
int i = 0;  
int j = s.length() - 1;  
while (i <= j) {  
    char l = s.charAt(i);  
    char r = s.charAt(j);  
    if (map.containsKey(l) &&  
        map.get(l) != r) {  
        return F;  
    } else {  
        i++;  
        j--;  
    }  
}  
return true;
```


○ Append Characters to String to Make Subsequence (2486)

Subsequence?

eg. babuadsa = String

b ~~o~~ ~~b~~ ~~p~~ ~~a~~ ~~d~~ ~~s~~ ~~a~~

b b d s

b a b u a ~~d~~ ~~s~~ a

b a b u a a

many more.

let eg

s = "coaching"

t = "coding"

↑ ↑ ~~↑~~ ~~↑~~ ~~↑~~ ~~↑~~

match

→ char at s $\neq$ char at t

⇒ i++ (to maintain the order)

→ char at s = char at t

⇒ i++ & j++

eg. s = coaching

↑

t = coding

login?

→ d not found in 's' toh 's' mein ding
append karva hai

becomes

s = coachingding

= append → 4

0 1 2 3 4 5 6 7 8
 C O ~~Q~~ ~~Q~~ ~~H~~ ~~I~~ ~~R~~ ~~G~~ ~~X~~
 ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
 i i i i i i i i

0 1 2 3 4 5
 C O d i n g
 ↑ ↑ ↑
 j j j

'i' par hogya
 and j at index

= 4

So,

Hame kitna
 append karna
 hai → "d~~o~~ing"

⇒ 4

⇒ 6 - 2 = 4

⇒ length - index

Code:

i = 0
 j = 0

while (i < s.length() & j < t.length()) {

if (s.charAt(i) == t.charAt(j)) {

j++
 i++

} else {

j++

}

TC = O(m + n)

SC = O(1)

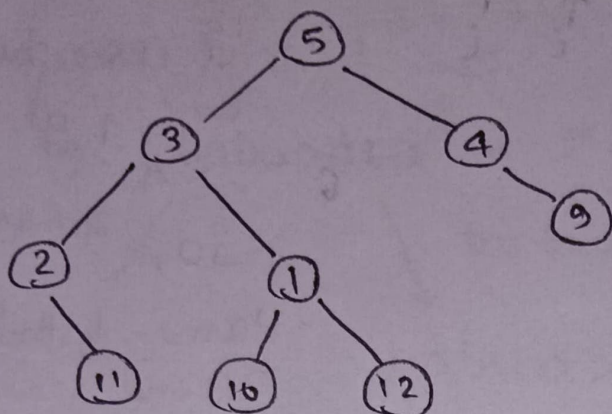
}

return t.length() - j

Q. Lowest Common Ancestor of a Binary Tree III

1650

eg.



Itself Descendants allowed.

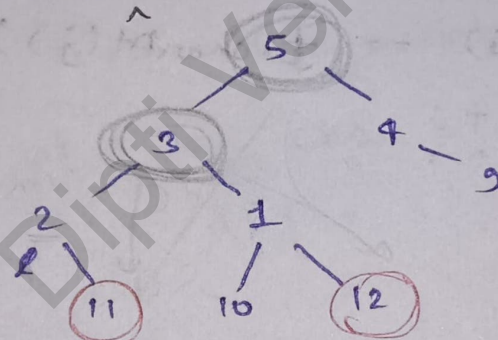
Node structure {
int value
Node left
Node right
Node parent
}

→ Input : 11 12

Output : 3

How?

Look hi kaha parent node mein 11 and 12 hai

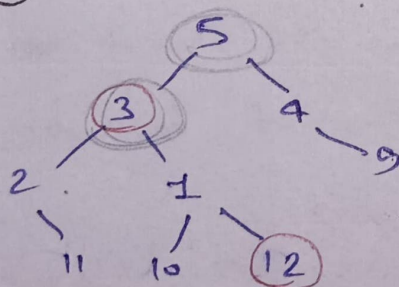


3 and 5 → But lowest = 3 → output.

→ Input : 3 12

Output : 3

How?

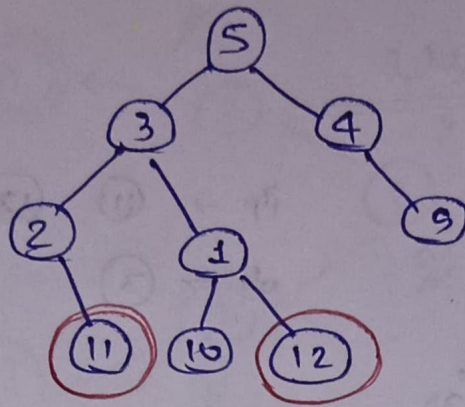


→ 12 ka parent ka parent = 3

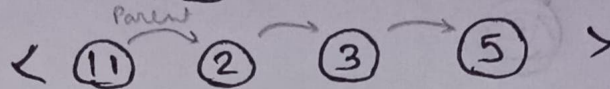
→ and question allowed itself descendants
so, ans = 3

3 and 5
→ lowest

Brute Force



Store the parent of 11 :



~~eye~~ (eye)
lookup

~~12~~ ~~10~~

Ab 12 node ke parents list ko 11th parent

list mein check karana hai

12 → not present

1 ×

3 ✓ (exists) ⇒ means this is the lowest common ancestor of both 11 & 12

Which data structure does this work fairly?

→ Set : insertion
lookup
deletion } $O(1)$

Code :

```
SetNode> set = new HashSet<>();
```

p = 11

q = 12

```
while (p != null) {  
    set.add(p)  
    p = p.parent  
}
```

set fill with < 2, 3, 5 >

```
while (q != null) {  
    if (set.contains(q)) {  
        return q  
    }
```

```
    q = q.parent  
}
```

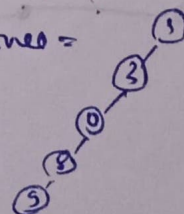
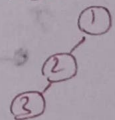
```
return null
```

TC = $O(n)$

SC = $O(n)$ → ?

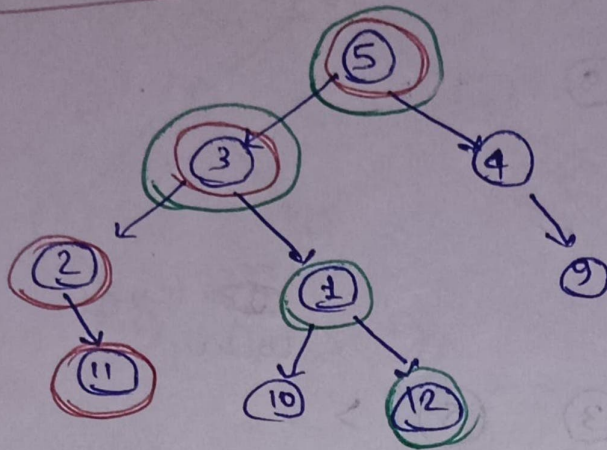
think : skew tree =

skew tree



< 9, 8, 0, 3, 1 > → store all → space = n

Optimized the space

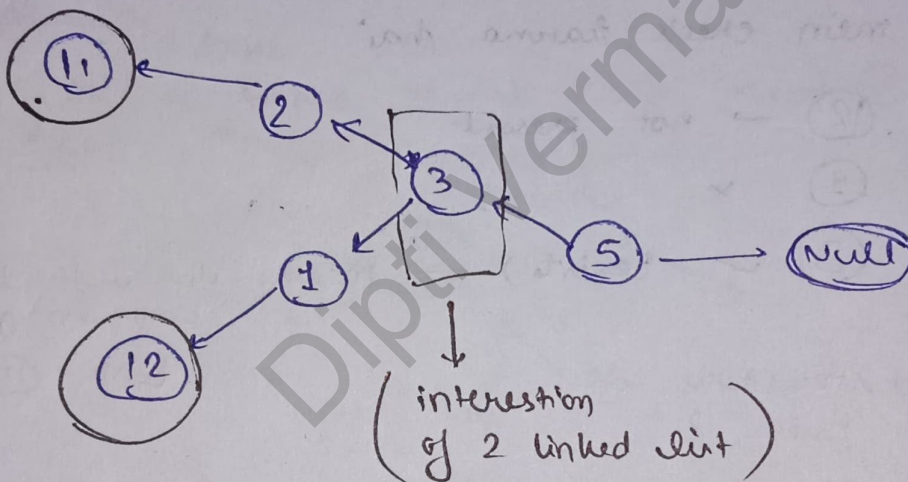


inp $\rightarrow$ 11 12

o/p $\Rightarrow$ 3

$\downarrow$ ~~root~~ rotate & ~~del~~ (remove extra node for understanding)

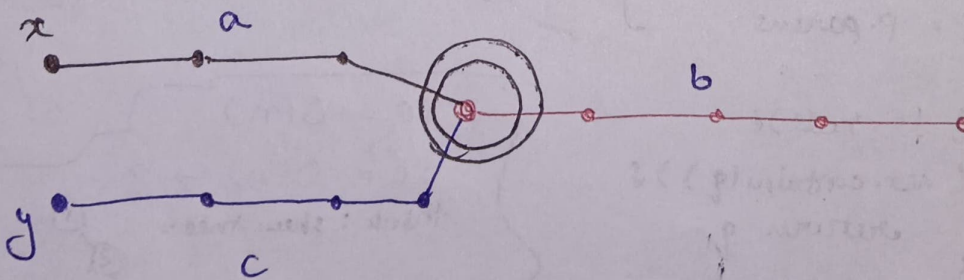
look
like
Linked
list



11. parent.parent = 3

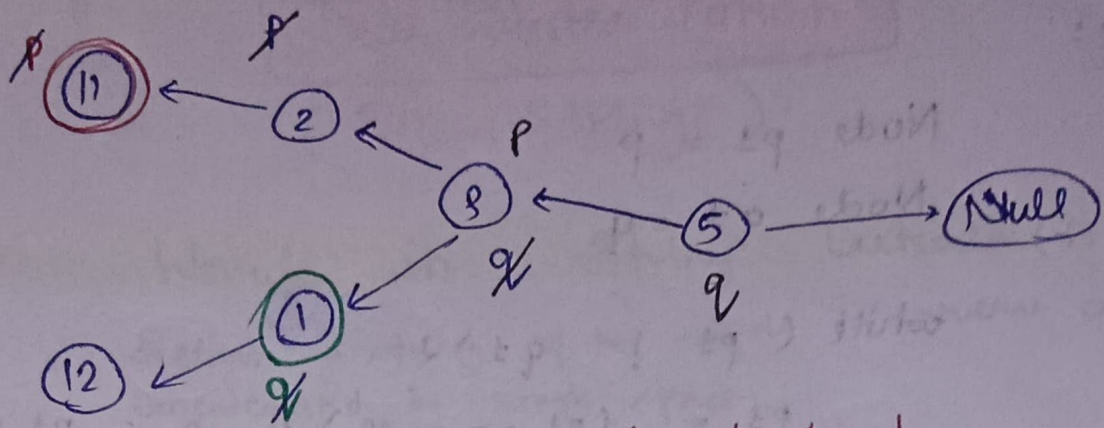
12. parent.parent = 3

Let $\rightarrow$ approach 2 runners on single track

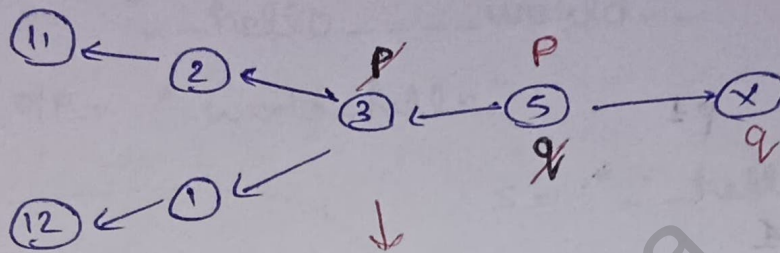


$x \rightarrow a + b + c$

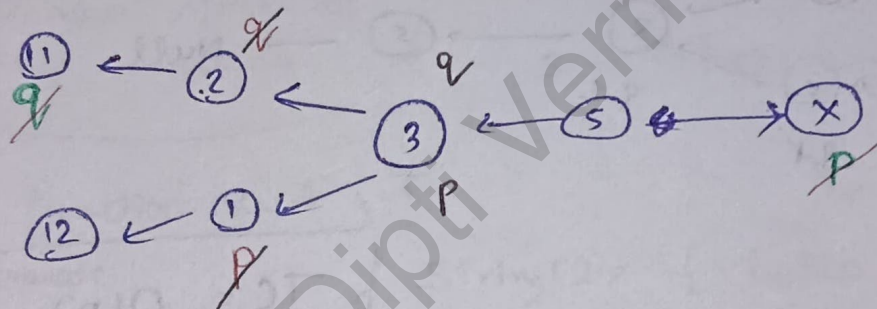
$y \rightarrow c + b + a$



↓ increment p & q by 1



(Now q → Null then in next iteration it moves on 11 node)



Now p and q both are at 3
i.e. Intersection.

→ Lowest Common Ancestor = 3 of p and q

Code:

Node $p1 = p$

Node $q1 = q$

while ($p1 \neq q1$)

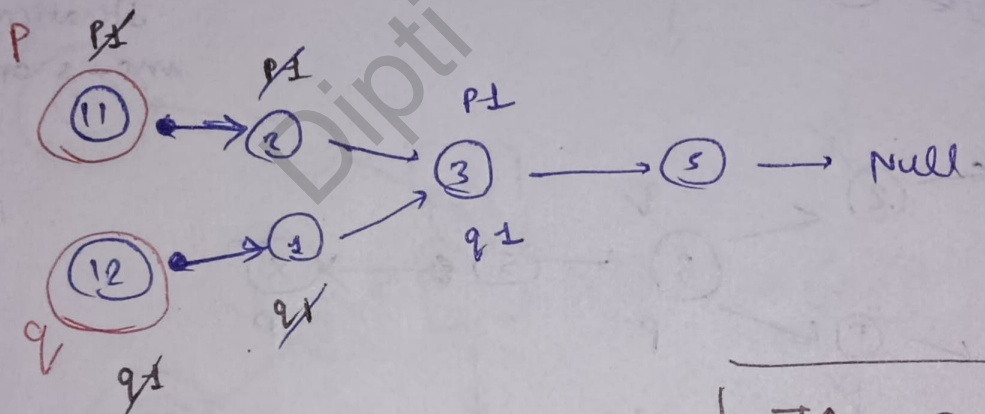
$p1 = (p1 == null) ? q : p1.parent$

$q1 = (q1 == null) ? p : q1.parent$

}

return $p1$

eg



→ return 3

TC = $O(n)$

SC = $O(1)$

Day-6

Two Pointer Pattern

(On String)

1. Reverse Words in a String Leetcode (151)

Return a string of the words in reverse order concatenated by single space.

Do not include any extra spaces.

Eg $s = \text{"--hello---world--"}$

o/p = "world_hello"

Brute Force

trim function used to remove spaces of starting & ending

$s = \text{"--hello---world--the--"}$

↓ trim

$s = \text{"hello---worldthe"}$

↓ split (1 space)

String() = [^{the}hello, world, ^{hello}the]
i j i

↓

String() = [the, world, hello]

↓ join (" - ")

"the world hello"

Split function used to separate and it returns in array

(join(", ") → it join the words

like =

"the, world, hello"

eg

Code:

```
String trim = s.trim()
String arr[] = trim.split("\\s+");
```

regular expression for
space
↑

```
i = 0, j = arr.length - 1
```

```
while (i < j) {
```

```
    temp = arr[i]
```

```
    arr[i] = arr[j]
```

```
    arr[j] = temp
```

```
    i++  
    j--
```

TC = $O(n)$

SC = $O(n)$

↓
bez, j

trim,
arr[]

```
return String.join(" ", arr)
```

In Java,

String → Immutable → Unchange

(Ek baar string ban gayi then, wo change nhi ho
skta, vo turant memory occupy kar leta hai.)

Eg, s = "babua dsa"

isme hum kuch bhi directly change nhi
kar skte.

But in java, there is concept of String Builder,
StringBuffer

s = "babua dsa"

↓ last character remove

babua ds

Approach II $\rightarrow$ (without using any function)

Think:

" babua - dsa "

↓ uneven

"asd - aubab"

↑ reverse ↓ reverse

2 dsa - babua"

Trick ★ ★

④ - Reverse String II

Reverse String III

eg, $s = \text{"hello world the"}$

↓ Step 1: remove starting & ending spaces

hello world the

Tab take string

(space check
prev character
ki space hai yeh
letter

String
builder

= [hello-world-the]

$$i \Rightarrow$$

↓ reverse ← j

[eht - drow - ollen] $\rightarrow$ ^{letter by} ~~word~~ reverse

$$f[eh\text{t} \ominus d\text{low} - \text{olleh}]$$

↑
i

↑ ↑ ↑ reverse
j j j

↑
j

↑ ↑ ↑ ↑ ↑ ↑ j (end)

j j j

Now (reverse i to $j-1$)

(the-world-hello)

Code 11 :

String s = "the sky is blue"

int i = 0, n = s.length() - 1

// remove starting space.

while (i < s.length()) {

if (s.charAt(i) == ' ') {

i++

} else {

break

}

// remove ending space

while (n >= 0) {

if (s.charAt(n) == ' ') {

n--

} else {

break

}

// Clear extra spaces b/w the words

StringBuilder sb = new StringBuilder();

while (i <= n) {

if (s.charAt(i) != ' ') {

sb.append(s.charAt(i))

i++

} else if (s.charAt(i) == ' ') {

if (sb.charAt(sb.length() - 1) != ' ') {

sb.append(' ')

i++

} else {

// ignore

i++

}

}

"the --- hello"

↓

"the_hello"

// Reverse

```
int i = 0, j = sB.length() - 1
```

```
while ( i < j ) {
```

```
    char temp = sB.charAt(i)
```

```
    sB.setcharAt(i, sB.charAt(j))
```

```
    sB.setCharAt(j, temp)
```

```
    i++
```

```
    j--
```

```
}
```

"the - hello"
↓
"eht - olleh"

// word by word reverse

```
int start = 0
```

```
end = 0
```

```
while ( start < sB.length() ) {
```

```
    while ( end < sB.length() && sB.charAt(end) != ' ' ) {
```

```
        end++
```

```
    }
```

```
    int p1 = start, p2 = end - 1
```

```
    while ( p1 < p2 ) {
```

```
        char temp = sB.charAt(p1)
```

```
        sB.setCharAt(p1, sB.charAt(p2))
```

```
        sB.setCharAt(p2, temp)
```

```
        p1++
```

```
        end p2--
```

```
    }
```

```
    }
```

```
    start = end + 1
```

```
    end = start
```

```
}
```

```
return sB.toString()
```

TC = $O(n)$

SC = $O(1)$

StringBuilders

"eht - olleh"

↓

"the - hello"